

GLOBALRAIN

Practices for Secure Software Report

Table of Contents

DOCUMENT REVISION HISTORY 3
CLIENT 3
INSTRUCTIONS 3
DEVELOPER 4
1. ALGORITHM CIPHER 4
2. CERTIFICATE GENERATION 4
3. DEPLOY CIPHER 4
4. SECURE COMMUNICATIONS 4
5. SECONDARY TESTING 5
6. FUNCTIONAL TESTING 6
7. SUMMARY 7
8. INDUSTRY STANDARD BEST PRACTICES 8

Document Revision History

Version	Date	Author	Comments
1.0	4/20/2025	Reagan Morgan	

Client



Instructions

Submit this completed practices for secure software report. Replace the bracketed text with the relevant information. You must document your process for writing secure communications and refactoring code that complies with software security testing protocols.

- Respond to the steps outlined below and include your findings.
- Respond using your own words. You may also choose to include images or supporting materials. If you include them, make certain to insert them in all the relevant locations in the document.
- Refer to the Project Two Guidelines and Rubric for more detailed instructions about each section of the template.

Developer
Reagan Morgan

1. Algorithm Cipher

I recommend SHA-256 because it is a cryptographic hash function that has no currently known collisions. I chose SHA-256 instead of the newer SHA3-256 because of its speed performance. However, in the future SHA3-256 may be a better option if, or when, SHA-256 develops known collisions and attacks.

2. Certificate Generation

Insert a screenshot below of the CER file.

```
C:\Program Files\Java\jdk-23\bin>keytool.exe -printcert -file serverpro2.cer
Owner: CN=Reagan Morgan, OU=SNHU, O=SNHU, L=Noble, ST=Oklahoma, C=US
Issuer: CN=Reagan Morgan, OU=SNHU, O=SNHU, L=Noble, ST=Oklahoma, C=US
Serial number: 16b536f521e38918
Valid from: Sun Apr 20 14:52:31 CDT 2025 until: Wed Apr 15 14:52:31 CDT 2026
Certificate fingerprints:
         SHA1: F8:BC:DC:5C:6B:06:EE:83:DA:D2:C6:56:9B:18:A3:AE:02:C2:15:B0
         SHA256: 64:A2:F4:E1:52:C2:A0:0E:9F:37:66:18:DC:A2:0B:CE:A7:7F:2C:AB:BA:85:A2:27:D7:7A:37:7A:FA:64:65:67
Signature algorithm name: SHA384withRSA
Subject Public Key Algorithm: 2048-bit RSA key
Version: 3

Extensions:

#1: ObjectId: 2.5.29.14 Criticality=false
SubjectKeyIdentifier [
KeyIdentifier [
0000: A8 1B 3D 4C D0 16 DB 00   6E E5 AD DB 45 5A 69 FF  ..=L....n...EZi.
0010: 97 02 79 A3                                        ..y.
]
]


C:\Program Files\Java\jdk-23\bin>
```

3. Deploy Cipher

Insert a screenshot below of the checksum verification.

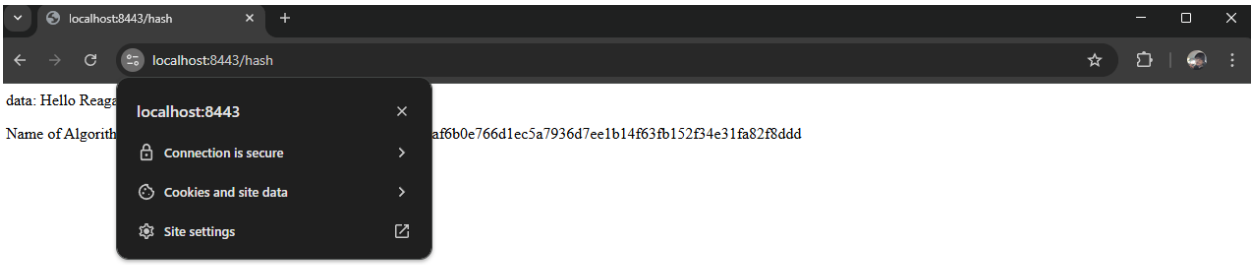


data: Hello Reagan Morgan!

Name of Algorithm: SHA-256 | CheckSum Value: 6891fe7b6d2936af6b0e766d1ec5a7936d7ee1b14f63fb152f34e31fa82f8ddd

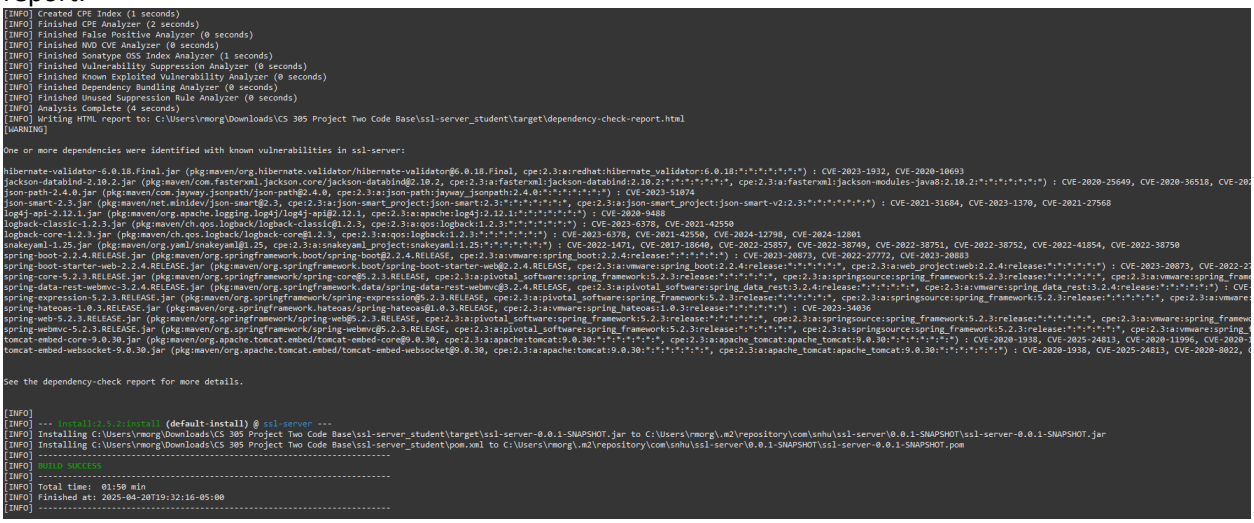
4. Secure Communications

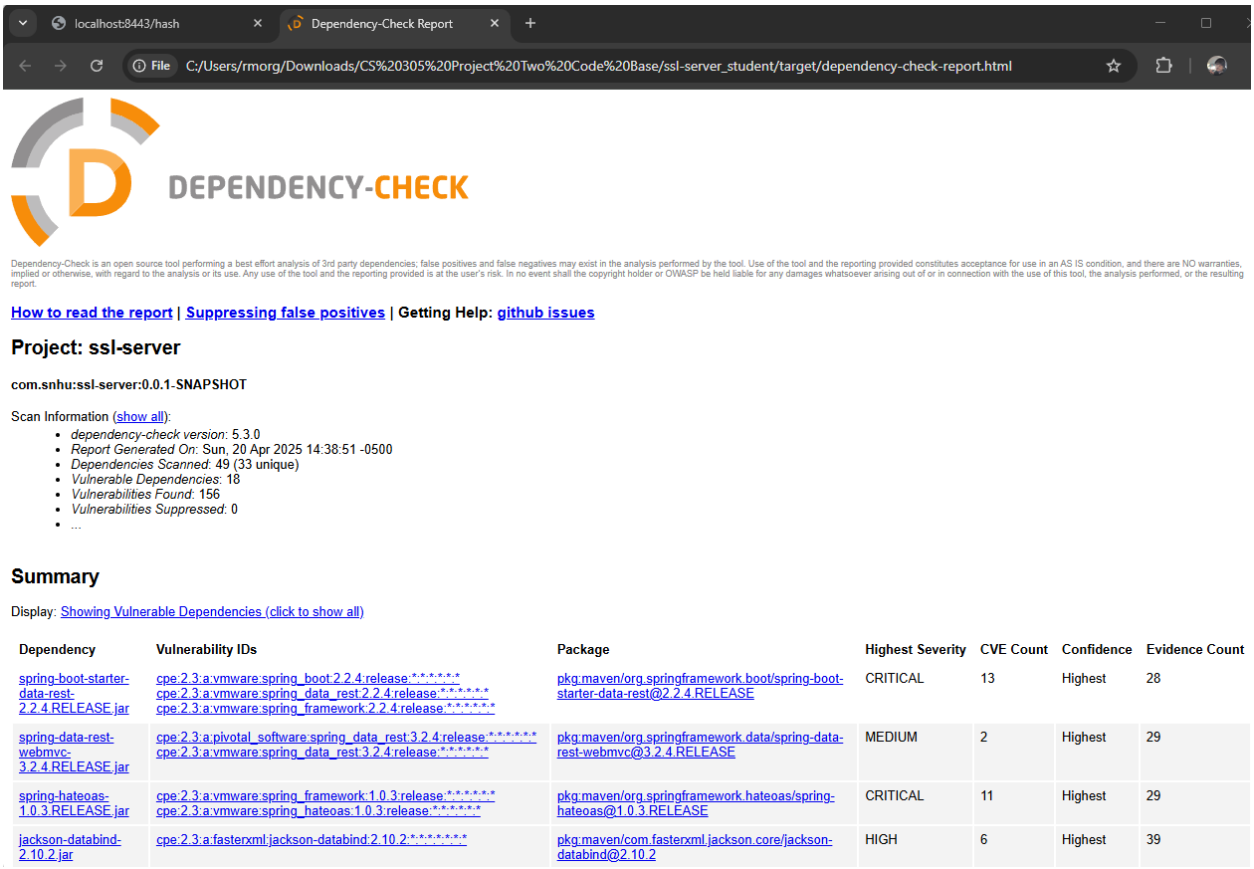
Insert a screenshot below of the web browser that shows a secure webpage.



5. Secondary Testing

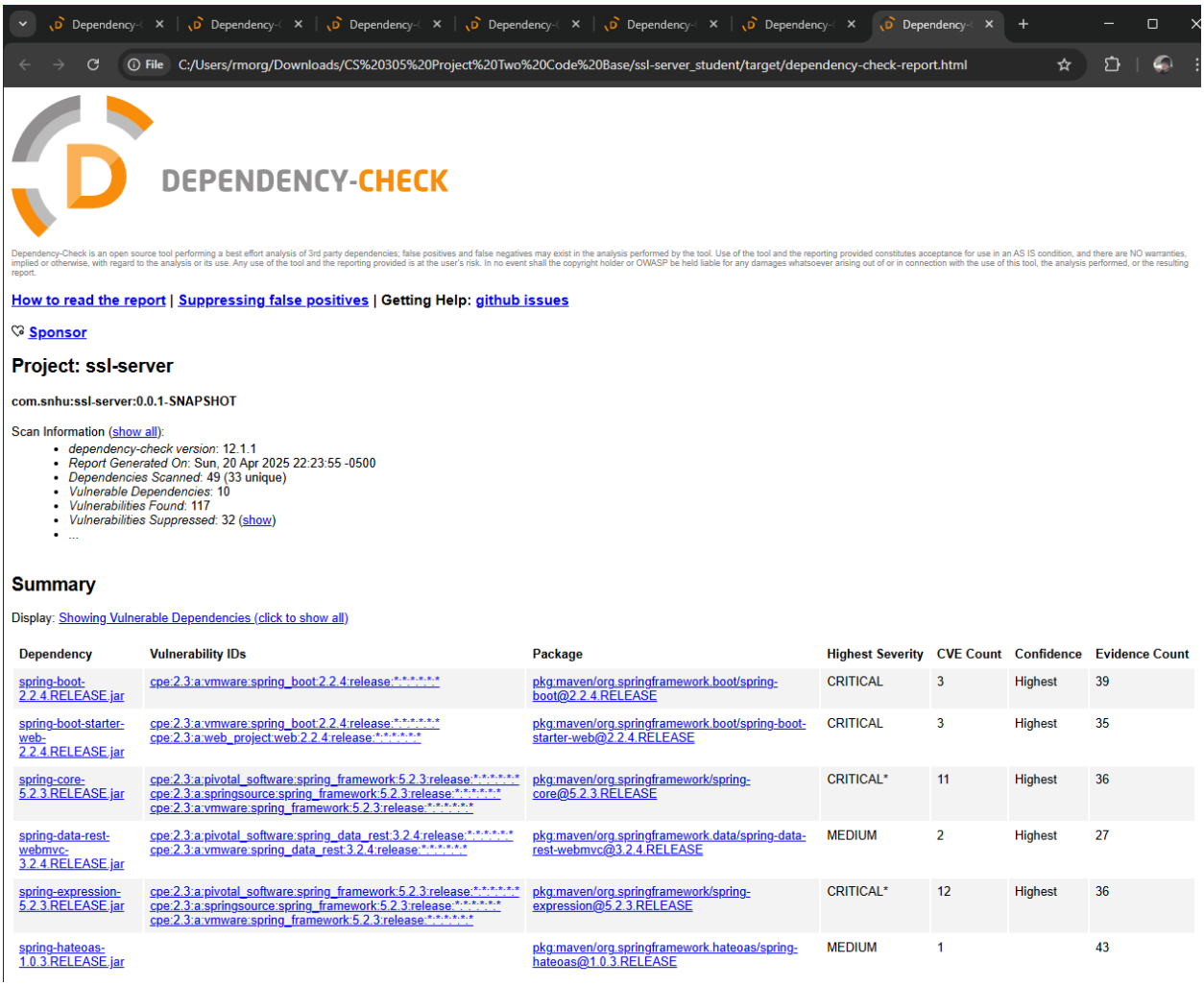
Insert screenshots below of the refactored code executed without errors and the dependency-check report.





6. Functional Testing

Insert a screenshot below of the refactored code executed without errors.



DEPENDENCY-CHECK

Dependency-Check is an open source tool performing a best effort analysis of 3rd party dependencies; false positives and false negatives may exist in the analysis performed by the tool. Use of the tool and the reporting provided constitutes acceptance for use in an AS IS condition, and there are NO warranties, implied or otherwise, with regard to the analysis or its use. Any use of the tool and the reporting provided is at the user's risk. In no event shall the copyright holder or OWASP be held liable for any damages whatsoever arising out of or in connection with the use of this tool, the analysis performed, or the resulting report.

How to read the report | Suppressing false positives | Getting Help: github issues

Sponsor

Project: ssl-server

com.snhu:ssl-server:0.0.1-SNAPSHOT

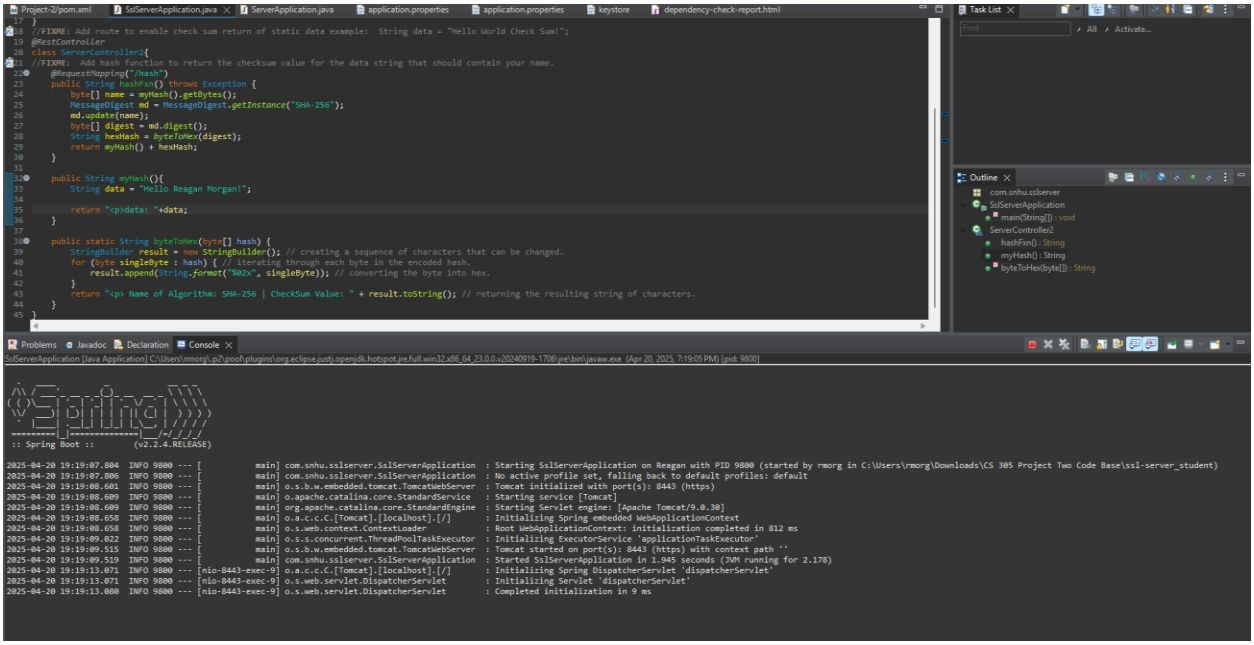
Scan Information (show all):

- *dependency-check version*: 12.1.1
- *Report Generated On*: Sun, 20 Apr 2025 22:23:55 -0500
- *Dependencies Scanned*: 49 (33 unique)
- *Vulnerable Dependencies*: 10
- *Vulnerabilities Found*: 117
- *Vulnerabilities Suppressed*: 32 (show)
- ...

Summary

Display: Showing Vulnerable Dependencies (click to show all)

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
spring-boot-2.2.4.RELEASE.jar	cpe:2.3:a:vmware:spring_boot:2.2.4:release:*:*:*:*:*:*	pkg:maven/org.springframework.boot/spring-boot@2.2.4.RELEASE	CRITICAL	3	Highest	39
spring-boot-starter-web-2.2.4.RELEASE.jar	cpe:2.3:a:vmware:spring_boot:2.2.4:release:*:*:*:*:*:* cpe:2.3:a:web_project:web:2.2.4:release:*:*:*:*:*:*	pkg:maven/org.springframework.boot/spring-boot-starter-web@2.2.4.RELEASE	CRITICAL	3	Highest	35
spring-core-5.2.3.RELEASE.jar	cpe:2.3:a:pivotal_software:spring_framework:5.2.3:release:*:*:*:*:*:* cpe:2.3:a:springsource:spring_framework:5.2.3:release:*:*:*:*:*:* cpe:2.3:a:vmware:spring_framework:5.2.3:release:*:*:*:*:*:*	pkg:maven/org.springframework/spring-core@5.2.3.RELEASE	CRITICAL*	11	Highest	36
spring-data-rest-webmvc-3.2.4.RELEASE.jar	cpe:2.3:a:pivotal_software:spring_data_rest:3.2.4:release:*:*:*:*:*:* cpe:2.3:a:vmware:spring_data_rest:3.2.4:release:*:*:*:*:*:*	pkg:maven/org.springframework.data/spring-data-rest-webmvc@3.2.4.RELEASE	MEDIUM	2	Highest	27
spring-expression-5.2.3.RELEASE.jar	cpe:2.3:a:pivotal_software:spring_framework:5.2.3:release:*:*:*:*:*:* cpe:2.3:a:springsource:spring_framework:5.2.3:release:*:*:*:*:*:* cpe:2.3:a:vmware:spring_framework:5.2.3:release:*:*:*:*:*:*	pkg:maven/org.springframework/spring-expression@5.2.3.RELEASE	CRITICAL*	12	Highest	36
spring-hateoas-1.0.3.RELEASE.jar		pkg:maven/org.springframework.hateoas/spring-hateoas@1.0.3.RELEASE	MEDIUM	1		43



```
 :: Spring Boot ::        (v2.2.4.RELEASE)

2025-04-20 19:19:07.804  INFO 9800 --- [           main] com.snhu.sslserver.SslServerApplication  : Starting SslServerApplication on Reagan with PID 9800 (started by rmorg in C:\Users\rmorg\Downloads\CS 305 Project Two Code Base\ssl-server_student)
2025-04-20 19:19:07.806  INFO 9800 --- [           main] com.snhu.sslserver.SslServerApplication  : No active profile set, falling back to default profiles: default
2025-04-20 19:19:08.601  INFO 9800 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat initialized with port(s): 8443 (https)
2025-04-20 19:19:08.609  INFO 9800 --- [           main] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
2025-04-20 19:19:08.609  INFO 9800 --- [           main] org.apache.catalina.core.StandardEngine  : Starting Servlet engine: [Apache Tomcat/9.0.30]
2025-04-20 19:19:08.658  INFO 9800 --- [           main] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring embedded WebApplicationContext
2025-04-20 19:19:08.658  INFO 9800 --- [           main] o.s.web.context.ContextLoader            : Root WebApplicationContext: initialization completed in 812 ms
2025-04-20 19:19:09.022  INFO 9800 --- [           main] o.s.s.concurrent.ThreadPoolTaskExecutor  : Initializing ExecutorService 'applicationTaskExecutor'
2025-04-20 19:19:09.515  INFO 9800 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port(s): 8443 (https) with context path ''
2025-04-20 19:19:09.519  INFO 9800 --- [           main] com.snhu.sslserver.SslServerApplication  : Started SslServerApplication in 1.945 seconds (JVM running for 2.178)
2025-04-20 19:19:13.071  INFO 9800 --- [nio-8443-exec-9] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring DispatcherServlet 'dispatcherServlet'
2025-04-20 19:19:13.071  INFO 9800 --- [nio-8443-exec-9] o.s.web.servlet.DispatcherServlet        : Initializing Servlet 'dispatcherServlet'
2025-04-20 19:19:13.080  INFO 9800 --- [nio-8443-exec-9] o.s.web.servlet.DispatcherServlet        : Completed initialization in 9 ms
```

7. Summary

For Artemis Finacial's website, I included a hash function to make data secure within their website since they are working with sensitive financial information. This covers the cryptography section of the vulnerability assessment flow chart. I also made sure that our connection was secure. This helps prevent DNS spoofing as well as eavesdroppers by encrypting data in transport as well. This covers the Client/Server section of the vulnerability assessment flow chart. Finally, I suppressed some vulnerabilities that do not relate to the website. However, I did leave in some that do not apply currently, but we do use the dependency listed. This can help future developers know what to avoid in case some things that do cause vulnerabilities are not implemented, like the issue with Cloud Foundry on the current version we use of Spring Boot. This covers the Code Error and APIs in the vulnerability assessment flow chart.

8. Industry Standard Best Practices

Throughout my code, I used industry standard best practices as defined by OWASP. In the suppression.xml file, I made sure to encode my output by using UTF-8. I made sure my passwords for the certificates were secure in order to lessen the impact of potential brute force attacks on our HTTPS certificate. The examples stated above in the summary are also examples of industry standard best practices when creating secure code. These standards are extremely important for all software to make sure it is safe for end users. It also makes sure your web applications and other software will act as expected, reducing the issues experienced by end users and making them more likely to come back to use your service. This applies to all software, not just those that handle sensitive information!